МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра интеллектуально информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-12

Выполнил:

Середич К. Н.

Студент группы ПО-12

Проверила:

Кулик А. Д.

Преподаватель-стажер

кафедры ИИТ

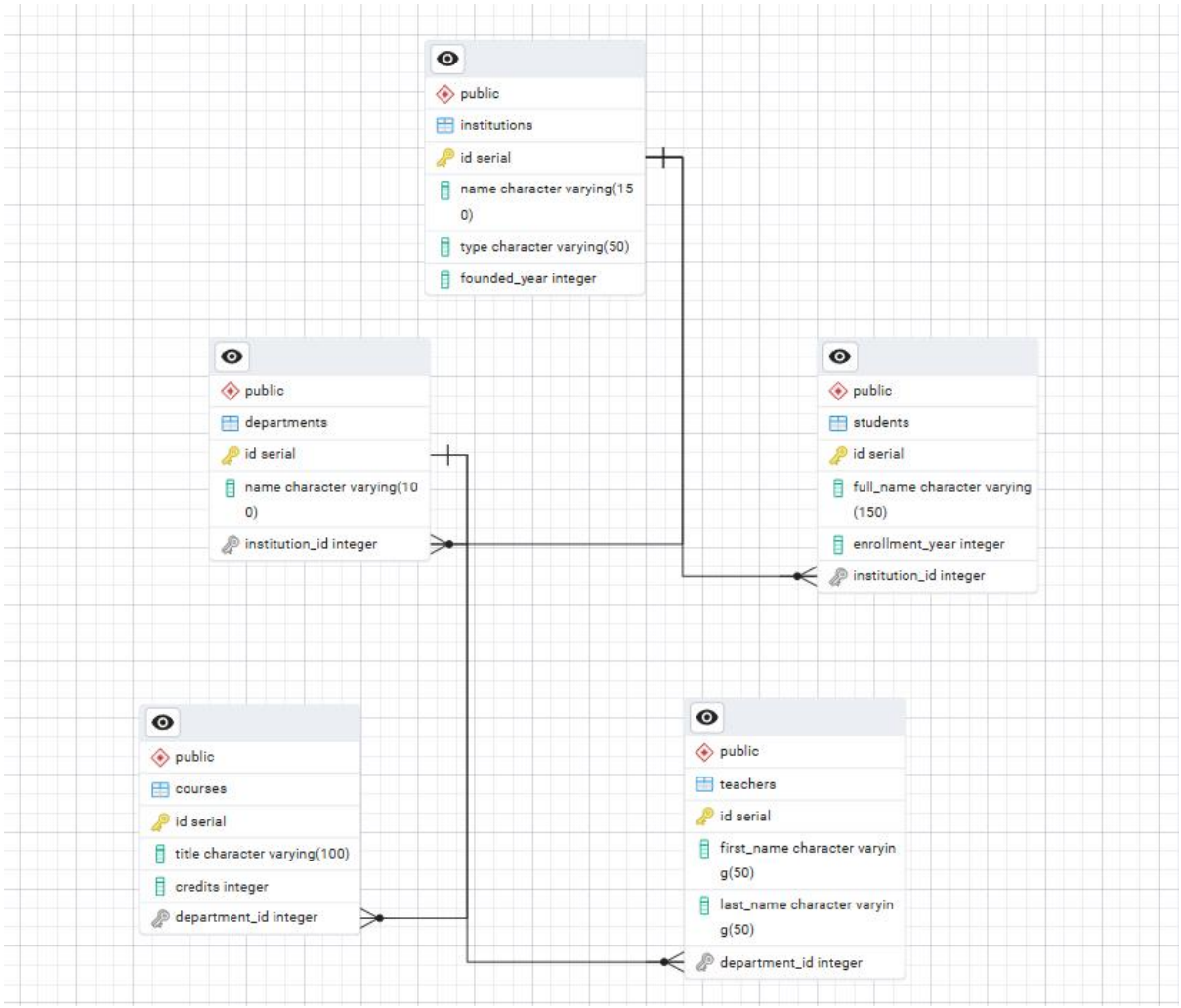
Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

21) База данных Образовательные учреждения



Спецификации ввода-вывода программы:

Для взаимодействия с программой используется REST API. Данные передаются и возвращаются в формате JSON.

1. Эндпойнт создания (POST /institutions/)
Ввод: JSON-объект, содержащий name (string), type (string), founded_year (integer).
Вывод: JSON-объект созданной записи с присвоенным id (integer).
2. Эндпойнт чтения (GET /institutions/)
Ввод: Параметры строки запроса (query) skip и limit для пагинации.
Вывод: Массив JSON-объектов со списком образовательных учреждений.
3. Эндпойнт обновления (PUT /institutions/{id})
Ввод: URL-параметр id. JSON-объект с обновленными данными name, type, founded_year.
Вывод: Обновленный JSON-объект учреждения. (При отсутствии — 404 Not Found).
4. Эндпойнт удаления (DELETE /institutions/{id})
Ввод: URL-параметр id.
Вывод: Сообщение об успешном удалении {"message": "..."}.

```python
"""
Модуль для работы с API образовательных учреждений.
Реализует подключение к PostgreSQL и CRUD операции с помощью FastAPI и SQLAlchemy.
"""

from typing import List, Generator

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey
from sqlalchemy.orm import declarative_base, sessionmaker, Session
from pydantic import BaseModel, ConfigDict


# Константы для подключения к БД
DATABASE_URL = "postgresql://postgres:333888@localhost:5432/education_db"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()


# --- SQLALCHEMY МОДЕЛИ ---


class Institution(Base):  # pylint: disable=too-few-public-methods
    """Модель образовательного учреждения."""

    __tablename__ = "institutions"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(150), nullable=False)
    type = Column(String(50))
    founded_year = Column(Integer)


class Department(Base):  # pylint: disable=too-few-public-methods
    """Модель факультета/отделения."""

    __tablename__ = "departments"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    institution_id = Column(Integer, ForeignKey("institutions.id", ondelete="CASCADE"))


class Teacher(Base):  # pylint: disable=too-few-public-methods
```

```python
    """Модель преподавателя."""

    __tablename__ = "teachers"
    id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    department_id = Column(Integer, ForeignKey("departments.id", ondelete="SET NULL"))


class Course(Base):  # pylint: disable=too-few-public-methods
    """Модель учебного курса."""

    __tablename__ = "courses"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(100), nullable=False)
    credits = Column(Integer)
    department_id = Column(Integer, ForeignKey("departments.id", ondelete="CASCADE"))


class Student(Base):  # pylint: disable=too-few-public-methods
    """Модель студента."""

    __tablename__ = "students"
    id = Column(Integer, primary_key=True, index=True)
    full_name = Column(String(150), nullable=False)
    enrollment_year = Column(Integer)
    institution_id = Column(Integer, ForeignKey("institutions.id", ondelete="CASCADE"))


# Создание таблиц
Base.metadata.create_all(bind=engine)


# --- PYDANTIC СХЕМЫ ---


class InstitutionBase(BaseModel):  # pylint: disable=too-few-public-methods
    """Базовая схема для учреждения."""

    name: str
    type: str
    founded_year: int


class InstitutionCreate(InstitutionBase):  # pylint: disable=too-few-public-methods
    """Схема для создания учреждения."""


class InstitutionResponse(InstitutionBase):  # pylint: disable=too-few-public-methods
    """Схема для ответа API (содержит ID)."""

    id: int
    model_config = ConfigDict(from_attributes=True)


# --- FASTAPI ПРИЛОЖЕНИЕ ---

app = FastAPI(title="Образовательные учреждения API")
```

```python
def get_db() -> Generator[Session, None, None]:
    """Генератор для получения сессии базы данных."""
    database_session = SessionLocal()
    try:
        yield database_session
    finally:
        database_session.close()


@app.post("/institutions/", response_model=InstitutionResponse)
def create_institution(inst: InstitutionCreate, database: Session = Depends(get_db)):
    """Создание нового образовательного учреждения."""
    db_institution = Institution(**inst.model_dump())
    database.add(db_institution)
    database.commit()
    database.refresh(db_institution)
    return db_institution


@app.get("/institutions/", response_model=List[InstitutionResponse])
def read_institutions(
    skip: int = 0, limit: int = 10, database: Session = Depends(get_db)
):
    """Получение списка образовательных учреждений с пагинацией."""
    return database.query(Institution).offset(skip).limit(limit).all()


@app.get("/institutions/{inst_id}", response_model=InstitutionResponse)
def read_institution(inst_id: int, database: Session = Depends(get_db)):
    """Получение информации об учреждении по его ID."""
    db_institution = (
        database.query(Institution).filter(Institution.id == inst_id).first()
    )
    if db_institution is None:
        raise HTTPException(status_code=404, detail="Institution not found")
    return db_institution


@app.put("/institutions/{inst_id}", response_model=InstitutionResponse)
def update_institution(
    inst_id: int, inst: InstitutionCreate, database: Session = Depends(get_db)
):
    """Обновление данных образовательного учреждения по его ID."""
    db_institution = (
        database.query(Institution).filter(Institution.id == inst_id).first()
    )
    if db_institution is None:
        raise HTTPException(status_code=404, detail="Institution not found")

    for key, value in inst.model_dump().items():
        setattr(db_institution, key, value)

    database.commit()
    database.refresh(db_institution)
    return db_institution


@app.delete("/institutions/{inst_id}")
def delete_institution(inst_id: int, database: Session = Depends(get_db)):
    """Удаление образовательного учреждения по его ID."""
```

```python
    db_institution = (
        database.query(Institution).filter(Institution.id == inst_id).first()
    )
    if db_institution is None:
        raise HTTPException(status_code=404, detail="Institution not found")

    database.delete(db_institution)
    database.commit()
    return {"message": "Учреждение успешно удалено"}
```

Вывод: приобрели практические навыки разработки API и баз данных.